Prerequisites and objectives

- Prerequisites are arrays, heaps, recursion, expectation, and Chapter 1 complexity definitions.
- Explain each algorithm's invariant, correctness, and cost. Distinguish stability, in-place storage, and the comparison model.
- Choose using existing order, key domain, memory limits, and worst-case requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sorting contract and stability

- The output must be a permutation of the input and satisfy `A[i]≤A[i+1]`. Order alone does not rule out lost or duplicated elements.
- Stability preserves the original relative order of equal-key records. Sorting `(2,A),(1,X),(2,B)` must leave A before B.
- Here in-place means O(1) extra data-array space. Recursion-stack space is reported separately.
- The comparator must express a consistent order. C++ sorting requires strict weak ordering. Using $\leq$ as an ordinary comparator violates that requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion-sort invariant

1. Before iteration i, `A[0,i)` is sorted and is a permutation of the original first i elements.
2. Save key=A[i]. Shift prefix elements strictly greater than key rightward until its insertion slot is available.
3. Write key there, obtaining sorted prefix `A[0,i+1)`.

Moving only strictly greater keys preserves stability. Example `[5,2,4,2]` becomes `[2,5,4,2]`, `[2,4,5,2]`, then `[2,2,4,5]`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion sort and inversions

- An inversion is a pair $i<j$ with $A[i]>A[j]$. Each rightward shift removes one inversion involving the current key.
- With I inversions, standard insertion sort takes $\Theta(n+I)$.
- Sorted input has $I=0$ and best time $\Theta(n)$. Distinct reverse-sorted input has $I=n(n-1)/2$ and worst time $\Theta(n^2)$.
- The inversion count explains suitability for small or nearly sorted inputs without an unsupported claim about average speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort partition contract

- Three-way partition divides the range around pivot value p into `<p`, `=p`, and `>p`. Recurse only on the outer regions.
- Maintain `[l,lt)<p`, `[lt,i)=p`, unclassified `[i,gt)`, and `[gt,r)>p`.
- For a smaller value, swap to lt and increment lt and i. For equality, increment i. For a larger value, decrement gt, swap there, and keep i to inspect the incoming value.
- The measure gt−i decreases each iteration. At termination every element is classified. Partition time is $\Theta(r-l)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-way partition indices and recursion contract

```
partition(A,l,r,p):
    lt=l; i=l; gt=r
    while i<gt:
        if A[i]<p: swap(A[lt],A[i]); lt+=1; i+=1
        else if A[i]>p: gt-=1; swap(A[i],A[gt])
        else: i+=1
    return (lt,gt)
quicksort(A,l,r):
    if r-l<2: return
    choose p from A[l:r]
    (lt,gt)=partition(A,l,r,p)
    quicksort(A,l,lt); quicksort(A,gt,r)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For [4,2,4,7,1,4], states (A, lt, i, gt) are initially (A,0,0,6), then (A,0,1,6), ([2,4,4,7,1,4],1,2,6), (same,1,3,6), ([2,4,4,4,1,7],1,3,5), (same,1,4,5), and ([2,1,4,4,4,7],2,5,5). Recurse only on [0,2) and [5,6).

Three-way partition indices and recursion contract (continued)

Choosing the pivot from the subarray guarantees a nonempty equal region and smaller recursive problems. An absent extreme pivot could leave a recursive side equal to the original range. Expected bounds require random selection, not a fixed middle-position value. Smaller-side recursion bounds stack depth but leaves quadratic worst-case time possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quicksort worst-case and expected time

- For `[4,2,4,7,1,4]` and p=4, a partition is `[2,1] | [4,4,4] | [7]`. Order inside regions is unspecified.
- With distinct keys and extreme pivots, `T(n)=T(n−1)+Θ(n)`, yielding worst-case $\Theta(n^2)$.
- For a fixed distinct-key input, uniformly random pivots in every subproblem give expected Θ(n log n). In the classic model comparing the pivot once with each other key, the exact expectation is `2(n+1)H_n−4n`. Randomness comes from the algorithm.
- Three-way partition removes equal keys together. The distinct-key exact formula does not count comparisons in every three-way implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expected quicksort comparisons and bounded recursion depth

```
quicksort_small_side(A,l,r):
    while r-l>=2:
        p=uniformly_random_value_from(A[l:r])
        (lt,gt)=partition(A,l,r,p)
        if lt-l<r-gt:
            quicksort_small_side(A,l,lt); l=gt
        else:
            quicksort_small_side(A,gt,r); r=lt
```

Rank the distinct keys $1, \dots, n$, and let X_{ij} indicate whether ranks i, j are ever compared directly. The classic two-way version compares each pivot exactly once with every other key in its subproblem. Two keys are compared exactly when the first pivot chosen from the rank interval $[i, j]$ is one of its endpoints. The $j - i + 1$ keys are symmetric, so $\Pr(X_{ij} = 1) = 2/(j - i + 1)$. By linearity of expectation,

$$\mathbb{E}[C] = \sum_{i<j} \frac{2}{j - i + 1} = 2 \sum_{d=1}^{n-1} \frac{n - d}{d + 1} = 2(n + 1)H_n - 4n = \Theta(n \log n),$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Expected quicksort comparisons and bounded recursion depth (continued)

where $H_n = \sum_{k=1}^{n} 1/k$.

The three-way implementation tests <, >, and = sequentially, so its distinct-key partitions still use only a bounded number of comparisons per key. Expected O(n log n) remains valid, but the exact classic comparison count does not transfer unchanged. Each pivot must be compared with every nonpivot key at least once. The same random rank splits therefore also give the indicator-sum lower bound, establishing expected Θ(n log n) for distinct keys. All-equal input needs only one three-way partition and Θ(n) work.

Space has a separate deterministic proof: the actual recursive child above is the smaller nonequal region, of length at most (r−l−1)/2. Active frame sizes therefore halve at each nesting, giving worst-case O(log n) frames. The larger side uses the current frame's while loop. Even repeated extreme pivots still scan total length n+(n−1)+…=Θ(n^2); bounded stack space does not remove that time worst case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion space and depth protection

- Recursing on both sides can have $\Theta(n)$ worst-case stack depth. In-place partition does not imply constant total auxiliary space.
- Recurse only on the smaller side and process the larger side in a loop. Each actual recursive subproblem is at most half as large, limiting stack space to $O(\log n)$.
- This improves space but does not eliminate $\Theta(n^2)$ worst-case comparison work.
- Introspective sorting additionally switches to heapsort when recursion becomes too deep. It is an engineering extension.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge sort and stable merging

- For sorted halves, compare their first unconsumed values and output the smaller. Choose the left value on equality to preserve stability.
- The output is the sorted merge of consumed input, and its last value is no larger than either unconsumed remainder.
- Merging `[2_A,5]` and `[2_B,4]` gives `[2_A,2_B,4,5]`. Taking right first on equality reverses the equal-key records.
- Standard array merging uses a Θ(n) buffer. The recurrence `T(n)=2T(n/2)+Θ(n)` gives Θ(n log n) worst-case time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete stable merge and merge-sort

```
merge(L,R):
    i=0; j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i].key<=R[j].key: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    append remaining L[i:]; append remaining R[j:]
    return out
merge_sort(A):
    if len(A)<2: return A
    m=len(A)//2
    return merge(merge_sort(A[:m]),merge_sort(A[m:]))
```

Merging L=[2_A,5], R=[2_B,4] first takes A on equality, with cursors (1,0) and output [A]. Taking B gives (1,1) and [A,B]. Taking 4 gives (1,2) and [A,B,4]. Appending 5 completes [A,B,4,5]. Choosing the smaller remaining head proves order, and consuming every input exactly once proves permutation preservation.

The slicing merge_sort is semantic pseudocode. The shared-buffer C++ implementation avoids retaining needless copies across levels.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Heapsort's sorted suffix

- For ascending order, build a max-heap, swap the root with the last heap element, and exclude that position from the heap.
- The prefix is a max-heap, the suffix is increasing, and every prefix key is at most every suffix key.
- The extracted root is the largest remaining key and belongs at the front of the sorted suffix. Sift down the replacement root.
- Construction is Θ(n). Extractions take Θ(n log n) worst-case total time. An iterative implementation uses O(1) extra space and is generally unstable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete heapsort driver and sorted-suffix trace

```
heap_sort(A):
    build_max_heap(A)
    for end=len(A)-1 down to 1:
        swap(A[0],A[end])
        sift_down_max(A,0,end)    // heap is [0,end), sorted suffix excluded
```

Heapsort on the same keys starts with max-heap [5,2,4,2]. Removing 5 gives [2,2,4|5], sifted to [4,2,2|5]. Removing 4 gives [2,2|4,5], then [2|2,4,5]. The fixed sorted suffix never reenters sifting. Equal value results do not establish record-identity stability.

Max-sifting chooses the larger child and stops when its parent is at least as large; indices match the min version with reversed comparisons.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Counting sort assumes a bounded key domain

- Integer keys in `[0,K)` use K counters. Prefix sums give the end position of each key's output block.
- Scan original records right to left, placing key k at `--end[k]`. This preserves equal-key order.
- Time and combined count/output space are Θ(n+K). A huge K relative to n may be impractical.
- Directly indexing by key obtains more than pairwise comparison information, so the pure comparison lower bound does not apply directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable counting allocation and record-by-record scatter

```
stable_count(records,K):
    out=array(len(records)); count=[0]*K
    for x in records: require 0<=x.key<K; count[x.key]+=1
    end=prefix_sum_inclusive(count)
    for x in reverse(records): end[x.key]-=1; out[end[x.key]]=x
    return out
```

For [(5,A),(2,B),(4,C),(2,D)],K=6, count=[0,0,2,0,1,1] and end=[0,0,2,2,3,4]. Right-to-left scattering sends D→out[1],C→out[2],B→out[0],A→out[3]. Output [B,D,C,A] preserves B before D. LSD passes require this stability, not an arbitrary swap-based equivalent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stability in LSD radix sort

- With base b and d digits, sort stably by each digit from least significant to most significant.
- After pass k, values are ordered by their lowest k digits. Stability in the next pass preserves lower-digit order within equal next digits.
- For `31,14,22,11`, the units pass gives `31,11,22,14`. A stable tens pass gives `11,14,22,31`.
- Stable counting sort costs $\Theta(n+b)$ per pass and $\Theta(d(n+b))$ overall. Signed values, variable-length strings, and Unicode need explicit encoding and ordering rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete LSD driver and digit-order invariant

```
LSD(A,b,d):
    for position=0..d-1:
        A=stable_count_by_digit(A,position,b)
    return A
```

LSD assumes fixed width d and base b≥2, digit=(key//b^position) mod b, and nonnegative or order-preservingly encoded keys.

For 31,14,22,11, the units digits are 1,4,2,1, so the stable first pass gives 31,11,22,14. The tens digits are then 3,1,2,1, giving 11,14,22,31. Stability preserves lower-digit order inside equal next digits, proving the low-k-digit invariant. Arbitrary-precision division would invalidate constant-time digit extraction in Θ(d(n+b)).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision-tree lower bound

- For n distinct keys, assume a deterministic algorithm learns order only by comparing two keys. Comparison histories form a binary decision tree.
- All n! relative input permutations need distinguishable leaves. A height-h binary tree has at most 2^h leaves, so $h \geq \lceil \log_2(n!) \rceil$.
- Since $n! \geq (n/2)^{n/2}$, we have $\log_2(n!) = \Omega(n \log n)$. Worst-case comparisons are therefore Ω(n log n).
- The model concerns distinct keys and arbitrary permutations. It neither forbids linear sorting of bounded-domain keys nor requires n log n comparisons on every input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserve qualifications in comparisons

Method	Best case	Worst case	Main auxiliary space	Stable
Insertion	$\Theta(n)$	$\Theta(n^2)$	$O(1)$	Yes, move only strictly greater keys
Randomized quicksort	Depends on partitions and duplicates	$\Theta(n^2)$, expected $\Theta(n \log n)$ for distinct keys	Stack can be $O(\log n)$	Generally no
Merge	$\Theta(n \log n)$, standard version here	$\Theta(n \log n)$	$\Theta(n)$	Yes, take left on equality
Heap	Depends on duplicates and implementation	$\Theta(n \log n)$	$O(1)$, iterative version	No
LSD radix	$\Theta(d(n+b))$, version here	Same	$\Theta(n+b)$	Every pass must be stable

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Preserve qualifications in comparisons (continued)

Assume constant-time key operations. Incompatible cost models do not establish which method is faster.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mistakes and verification

- Check order, the element multiset, and record stability where promised. is_sorted alone can miss data loss.
- Equal keys, sorted and reverse-sorted inputs, empty and singleton arrays, and many duplicates expose different partition boundaries.
- `examples/sorting.cpp` compares original insertion, merge, and three-way quicksort with standard-sort results on fixed-seed data, including negative values and duplicates.
- Executable cases are not a proof. Preserve partition invariants and recursive termination arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and extensions

- The MIT 6.006 2008 index, L10–L11, identifies course material on lower bounds and noncomparison sorting.
- Comparator requirements were checked against C++ sorting requirements. An interface requirement does not mandate one fixed implementation algorithm.
- Inversion explanations, partition traces, data, and exercises are original. Introsort is an explicit extension.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study